Exercise 1 - More on functions

In this exercise you will practise what we saw in the lecture:

- **Part A** – `return`, keyword arguments and default arguments
- **Part B** – a small movie picker for the IMDB data

Beginners: Do Part A first, then Part B. Use the hints in `Exercise 1 – step by step – day 4` whenever you get stuck, and please ask if something is unclear.

Advanced: Try to solve everything without the hints, and then do the extra tasks at the end.

Part A - Warm-up

A1. `print` vs `return`

The function below counts the `G` and `C` in a DNA sequence, but it only **prints** the result.

In [ ]:
```
def print_gc_count(seq):
    count = 0
    for base in seq:
        if base == 'G' or base == 'C':
            count += 1
    print(count)

print_gc_count('ATGC')
```

1. Write a new function `gc_count(seq)` that **returns** the count instead of printing it.
2. Use your function to calculate the **total number** of `G` and `C` in the two sequences `'ATGC'` and `'GGCC'`.

In [ ]:
```

```

A2. Return more than one value

Write a function `count_at_gc(seq)` that returns **two** values: the number of `A` + `T` and the number of `G` + `C`.

You can use the function you created in **A1** as a starting point.

The final version of the function should work like this:

```
at, gc = count_at_gc('ATGCGC')
print(at, gc)
```

```
   2 4
```

In [ ]:

A3. Positional and keyword arguments

Here is a function with three parameters:

In [ ]:
```
def describe_gene(name, chrom, start):
    print(f"{name} is on {chrom} and starts at position {start}")
```

Call `describe_gene` for the gene `BRCA1`, on `chr17`, starting at `43044295`:

1. with **positional** arguments only
2. with **keyword** arguments only, in a different order than in the definition
3. with **both** positional and keyword arguments

In [ ]:

Now run the cell below. Why does it give an error?

In [ ]:
```
describe_gene(name='BRCA1', 'chr17', 43044295)
```

A4. Default arguments

In the lecture, `cytosine_count` could only count `C`:

```
def cytosine_count(nucleotides):
    count = 0
    for x in nucleotides:
        if x == 'c' or x == 'C':
            count += 1
    return count
```

Write a function `count_base(seq, base='C')` that counts any base, with `'C'` as the **default**.

```
count_base('ACCG')            # should return 2
count_base('ACCG', base='G')  # should return 1
```

In [ ]:

Part B - IMDB movie picker

Create a function `pick_movie` that **returns** the title of the first movie in `250.imdb` that matches the user's wishes. The user can give **one or more** of these keyword arguments:

- `genre`
- `year`
- `rating_min` (the movie's rating must be at least this value)

If the user does not give a value for an argument, it should be `None`. Start with this definition:

```
def pick_movie(genre=None, year=None, rating_min=None):
```

If no movie matches, the function should return `None`.

Feel free to reuse code from previous days for reading the file!

Build it step by step and test after each step:

1. Only `genre`: `pick_movie(genre="Drama")` should return `'Paths of Glory'`
2. Add `year`: `pick_movie(year=2009, genre="Mystery")` should return `'The Secret in Their Eyes'`
3. Add `rating_min`: `pick_movie(rating_min=9)` should return `'Dangal'`
4. No match: `pick_movie(genre="Horror", year=2001)` should return `None`

In [ ]: